

Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Lista de Exercícios

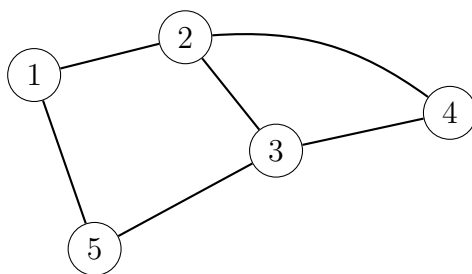
Código da Aula: 4

Leia as instruções antes de fazer a lista:

Não é necessário testar se os dados passados por argumento são válidos a menos que pedido na questão.

- Q1.** Em um parque de diversões alguns brinquedos só podem ser utilizados por pessoas acima de uma determinada altura. Escreva uma função em Python que recebe por argumento, nesta ordem, uma lista de números inteiros ou floats que representam a altura das pessoas na fila, e um número, que representa a altura mínima necessária para o brinquedo. Sua função deve retornar uma nova lista com a altura somente das pessoas que podem utilizar o brinquedo.
- Q2.** Um sensor ambiental registra leituras de temperatura ao longo do dia em uma lista. Devido a falhas técnicas, algumas leituras registradas são valores negativos inválidos. Escreva uma função em Python que receba por argumento uma lista de temperaturas e remova todas as leituras negativas da própria lista original utilizando o método `.pop()`. A função deve retornar o valor médio das temperaturas válidas restantes utilizando `sum()` e `len()`. Se a lista não contiver nenhuma temperatura válida ao final, retorne `0.0`.
- Q3.** Em uma competição de ginástica artística, a pontuação final de cada atleta é avaliada por um painel de 10 juízes. Para garantir uma avaliação justa, a regra oficial exige descartar apenas uma ocorrência da menor nota e apenas uma ocorrência da maior nota atribuída, somando-se as 8 notas restantes. Escreva uma função em Python que receba por argumento uma lista de números flutuantes (`float`) representando as notas de um atleta. A função deve calcular e retornar a soma das notas válidas. A função deve validar o tamanho da entrada, retornando o valor `-1` caso a lista contendo as notas não possua exatamente 10 elementos. Em caso de empate nas notas extremas (por exemplo, dois juízes atribuírem a nota mínima), apenas uma dessas notas deve ser descartada. É obrigatório o uso das funções `sort()`, `sum()` e `len()`.
- Q4.** Escreva uma função em Python que recebe duas listas de números inteiros e de mesmo tamanho e retorna uma nova lista na qual, para cada posição, se o elemento daquela posição da primeira lista for igual ao elemento da mesma posição na segunda lista, o valor na lista retornada deve ser 1; se o elemento daquela posição da primeira lista for diferente do elemento da mesma posição da segunda lista, mas este elemento estiver contido na segunda, o valor na lista retornada deve ser 2; se o elemento daquela posição da primeira lista for diferente do elemento da mesma posição da segunda lista e este elemento não estiver contido na segunda, o valor na lista retornada deve ser 0. É permitido o uso das funções `range()` e `len()` nesta questão. Se as duas listas tiverem tamanhos diferentes, a função deve retornar `-1`.

- Q5.** Escreva uma função em Python que recebe uma lista de listas e retorna o somatório dos produtos dos elementos de cada sublista. Por exemplo, para a entrada `[[2,4],[5,2,10]]`, a saída deve ser 108, pois $2 \times 4 + 5 \times 2 \times 10 = 108$. Para essa questão, não utilize a função `range()`.
- Q6.** Escreva uma função em Python que receba por argumento uma lista de números (`int` ou `float`). A função deve verificar a quantidade de elementos da lista e, caso ela esteja vazia ou contenha uma quantidade ímpar de elementos, deve retornar o código de erro `-1`. Caso a lista contenha uma quantidade par de elementos, a função deve calcular a média dos valores da lista original, inverter a ordem dos elementos da lista utilizando o método `.reverse()` e dividi-la em duas partes iguais. Para entradas válidas, a função deve retornar explicitamente uma saída com 3 valores, nesta exata ordem: `media`, `lista_primeira_metade`, `lista_segunda_metade`.
- Q7.** (Inspirada na OBI 2023 – Nível 2, Fase 1) Senhor Toupeira é o prefeito de Morro Seco e ao longo dos anos mandou construir muitos túneis embaixo da terra, conectando salões de convivência que ele também mandou construir, para alegria de sua comunidade de toupeiras. Cada túnel conecta exatamente dois salões de convivência distintos e não há dois túneis conectando o mesmo par de salões. Túneis podem ser usados em ambas as direções, ou seja, o túnel que conecta os salões A e B pode ser usado para ir da A para B ou de B para A. Salões de convivência possuem identificadores únicos. Senhor Toupeira agora quer incentivar que as toupeiras de Morro Seco façam caminhadas, para melhorar a saúde da comunidade. Para isso preparou um caderno com várias sugestões de passeio pelos túneis e salões de convivência, em que cada sugestão de passeio é descrita como uma sequência de salões de convivência, que devem ser visitados estritamente na ordem dada. No entanto, Senhor Toupeira foi alertado de que algumas das sugestões de passeio estão incorretas, pois não são possíveis. A figura abaixo mostra um exemplo de salões de convivência e túneis, em que salões têm identificadores 1, 2, 3, 4 e 5.



Um passeio composto pela sequência de salões $\{5, 3, 4, 3, 2\}$ é possível. Mas o passeio composto pela sequência de salões $\{2, 3, 5, 4\}$ não é possível, pois não existe túnel entre os salões 5 e 4. Sua tarefa é escrever uma função em Python que recebe por argumento uma lista de listas que indica as conexões entre os salões e outra lista de listas com possíveis passeios, e retorna um número inteiro que informa a quantidade de passeios que são possíveis. O primeiro argumento é uma lista de listas onde cada sublista contém somente dois elementos que indicam as conexões entre dois salões. Por exemplo, a figura acima pode ser representada por `[[1, 2], [1, 5], [2, 3], [3, 5], [2, 4], [4, 3]]`, mas a ordem dos números é indiferente, de forma que as listas de listas `[[2, 1], [5, 1], [3, 2], [3, 5], [4, 2], [4, 3]]` e `[[2, 3], [3, 4], [4, 2], [3, 5], [1, 5], [1, 2]]` também representam as conexões

dessa mesma figura. O segundo argumento é uma lista de listas onde cada sublista representa um possível passeio. Por exemplo, a lista `[[5, 3, 4, 3, 2], [2, 3, 5, 4]]` indica dois possíveis passeios (que são os passeios do exemplo acima). Para essas entradas, a saída deveria ser 1, pois somente um dos dois passeios indicados é possível.